哈希表实验报告

1. 问题的描述

本实验的目的是实现两种常见的哈希表结构：线性探测哈希表和链表哈希表，并进行测试。哈希表用于解决冲突问题，允许快速查找、插入和删除操作。实验的目标是：

- 实现线性探测和链表哈希表。
- 使用哈希函数对给定的关键字进行哈希映射。
- 支持插入操作和查找操作，并计算相应的查找次数。
- 比较不同哈希表结构在查找性能上的差异。

输入数据：

1. 哈希表的大小（表的大小）。
2. 哈希函数的模数（p）。
3. 关键字数量（n），以及n个要插入的关键字。

输出结果：

1. 哈希表的状态，显示每个槽位的内容。
2. 每个关键字的查找成功次数与失败次数。
3. 查找成功和失败的平均查找长度。

2. 算法的描述

(1) 数据结构的描述

线性探测哈希表

- **存储结构**：使用一个一维数组 `table[]` 存储哈希表的内容，另外用一个布尔型数组 `occupied[]` 来标记每个槽是否被占用。
- **主要变量**：
  - `table[]`：存储插入的关键字。
  - `occupied[]`：标记每个槽是否已经被占用，避免重复插入。
  - `size`：哈希表的大小。
  - `p`：哈希函数的模数，用于确定插入位置。

链表哈希表

- **存储结构**：使用一个指向节点结构体的数组 `table[]`。每个槽位是一个链表，链表节点包含一个关键字以及指向下一个节点的指针。
- **主要变量**：
  - `table[]`：存储链表头指针的数组。
  - `Node` 结构体：每个节点包含关键字 `key` 和指向下一个节点的指针 `next`。
  - `size`：哈希表的大小。
  - `p`：哈希函数的模数，用于确定插入位置。

(2) 程序结构的描述

函数原型与功能

- `int hashfunction(int key, int p)`：哈希函数，输入一个关键字 `key` 和模数 `p`，返回哈希值，即 `key % p`。
- **HashTableLinear 类**：
  - `insert(int key)`：插入一个关键字到哈希表中，使用线性探测解决冲突。
  - `search(int key, bool& success)`：查找一个关键字，返回查找是否成功以及探测次数。
  - `printTable()`：打印哈希表的内容以及每个槽的探测次数。
- **HashTableChaining 类**：
  - `insert(int key)`：插入一个关键字到哈希表中，使用链表解决冲突。
  - `search(int key, bool& success)`：查找一个关键字，返回查找是否成功以及探测次数。
  - `printTable()`：打印哈希表的内容和每个槽的探测次数。

全部代码：

```
#include <iostream>
using namespace std;

// 哈希函数
int hashfunction(int key, int p) {
    return key % p;
}

// 线性探测哈希表
class HashTableLinear {
private:
    int* table;      // 存储数据
    bool* occupied; // 当前位置是否被占用
    int size;        // 表的大小
    int p;           // 哈希表的模数
```

```
public:
    // 构造函数
    HashTableLinear(int TableSize, int mod): size(TableSize), p(mod) {
        table = new int[size];
        occupied = new bool[size];

        // 初始化哈希表
        for (int i = 0; i < size; i++) {
            table[i] = -1;          // -1 表示空槽
            occupied[i] = false;
        }
    }

    // 插入元素
    void insert(int key) {
        int index = hashfunction(key, p);
        while (occupied[index]) {
            index = (index + 1) % size;
        }
        table[index] = key;
        occupied[index] = true;
    }

    // 查找元素，返回成功与否以及探测次数
    int search(int key, bool& success) {
        int index = hashfunction(key, p);
        int probes = 0;
        while (probes < size) {
            probes++;
            if (!occupied[index]) {
                success = false;
                return probes; // 查找失败
            }
            if (table[index] == key) {
                success = true;
                return probes; // 查找成功
            }
            index = (index + 1) % size;
        }
        success = false;
        return probes;
    }

    // 计算失败查找次数：对于有关键字的失败查找，查找会遇到空槽
    int getFailureProbes(int key) {
        int index = hashfunction(key, p);
        int probes = 0;
        while (probes < size) {
            probes++;
```

```
        if (!occupied[index]) {
            return probes;  // 遇到空槽，返回失败探测次数
        }
        index = (index + 1) % size;
    }
    return probes;
}

// 打印哈希表并计算探测次数
void printTable() {
    cout << "哈希表的地址:\t";
    for (int i = 0; i < size; i++) {
        cout << i << "\t";
    }
    cout << endl;

    cout << "表中的关键字:\t";
    for (int i = 0; i < size; i++) {
        if (occupied[i]) {
            cout << table[i] << "\t";
        } else {
            cout << "-\t";
        }
    }
    cout << endl;

    // 初始化成功和失败查找次数数组
    int* successProbes = new int[size]();
    int* failureProbes = new int[size]();

    // 计算每个元素的成功查找次数
    for (int i = 0; i < size; i++) {
        if (occupied[i]) {
            bool success;
            successProbes[i] = search(table[i], success);
        }
    }

    // 计算每个元素的失败查找次数
    for (int i = 0; i < size; i++) {
        if (occupied[i]) {
            failureProbes[i] = size - i - 1;
        } else {
            // 空槽的失败查找次数为 1
            failureProbes[i] = 1;
        }
    }

    // 输出探测次数
```

```
        cout << "成功查找次数:\t";
        for (int i = 0; i < size; i++) {
            cout << successProbes[i] << "\t";
        }
        cout << endl;

        cout << "失败查找次数:\t";
        for (int i = 0; i < size; i++) {
            cout << failureProbes[i] << "\t";
        }
        cout << endl;

        // 计算平均查找长度
        double successTotal = 0, failureTotal = 0;
        int successCount = 0, failureCount = 0;

        for (int i = 0; i < size; i++) {
            if (occupied[i]) {
                successTotal += successProbes[i];
                successCount++;
            }
            failureTotal += failureProbes[i];
            failureCount++;
        }

        cout << "查找成功的平均查找长度: "
             << (successCount > 0 ? successTotal / successCount : 0) << endl;
        cout << "查找失败的平均查找长度: "
             << (failureCount > 0 ? failureTotal / failureCount : 0) << endl;

        delete[] successProbes;
        delete[] failureProbes;
    }
};

// 链表Hash
class HashTableChaining {
private:
    struct Node {
        int key;
        Node* next;
        Node(int k): key(k), next(nullptr) {}
    };

    Node** table;
    int size;
    int p;

public:
```

```
    HashTableChaining(int TableSize, int mod): size(TableSize), p(mod) {
        table = new Node*[size];
        for (int i = 0; i < size; i++) {
            table[i] = nullptr;
        }
    }

    void insert(int key) {
        int index = hashfunction(key, p);
        Node* newNode = new Node(key);
        newNode->next = table[index];
        table[index] = newNode;

    }

    int search(int key, bool& success) {
        int index = hashfunction(key, p);
        Node* current = table[index];
        int probes = 0;
        while (probes < size) {
            probes++;
            if (!current) {
                success = false;
                return probes;
            }
            if (current->key == key) {
                success = true;
                return probes;
            }
            current = current->next;

        }
        success = false;
        return probes;
    }
    void printTable() {
        cout << "哈希表的地址:\t";
        cout << "表中的关键字:\t";
        cout << endl;
        for (int i = 0; i < size; i++) {
            cout << i << "\t";
            Node* current = table[i];
            if (current) {
                while (current) {
                    cout << current->key << " ";
                    current = current->next;
                }
                cout << "\t";
            } else {
```

```
            cout << "-\t";
        }
        cout << endl;
    }
    cout << endl;


    // 初始化探测次数数组
    int* successProbes = new int[size];
    int* failureProbes = new int[size];
    for (int i = 0; i < size; i++) {
        successProbes[i] = 0; // 初始化为 0
        failureProbes[i] = 0; // 初始化为 0
    }

    // 计算每个槽位的成功和失败查找次数
    for (int i = 0; i < size; i++) {
        Node* current = table[i];
        int chainLength = 0; // 当前槽位链表长度
        while (current) {
            chainLength++;
            cout << current->key << "对应成功查找次数:\t" << chainLength << endl;
            successProbes[i] += chainLength; // 累加成功查找的探测次数

            current = current->next;
    }
        failureProbes[i] = chainLength + 1; // 失败查找的探测次数 = 链表长度 + 1
    }
    cout << endl;
    cout << "失败查找次数:\t";
    for (int i = 0; i < size; i++) {
    cout << failureProbes[i] << "\t";
}
    cout << endl;

    // 计算成功和失败的平均查找长度
    double successTotal = 0, failureTotal = 0;
    int successCount = 0, failureCount = size;

    for (int i = 0; i < size; i++) {
    if (table[i]) {
        successTotal += successProbes[i];
        Node* current = table[i];
        while (current) {
            successCount++; // 统计总的成功查找数
            current = current->next;
        }
    }
    failureTotal += failureProbes[i];
```

```
    }

        cout << "查找成功的平均查找长度: " << (successCount > 0 ? successTotal /
successCount : 0) << endl;
        cout << "查找失败的平均查找长度: " << (failureCount > 0 ? failureTotal /
failureCount : 0) << endl;

        delete[] successProbes;
        delete[] failureProbes;
}

};


// 主函数
int main() {
    int tableSize, mod, numKeys;
    cout << "请输入哈希表大小: ";
    cin >> tableSize;
    cout << "请输入模数: ";
    cin >> mod;
    cout << "请输入关键字数量: ";
    cin >> numKeys;

    HashTableLinear hashTable1(tableSize, mod);
    HashTableChaining hashTable2(tableSize, mod);

    cout << "请输入关键字: ";
    for (int i = 0; i < numKeys; i++) {
        int key;
        cin >> key;
        hashTable1.insert(key);
        hashTable2.insert(key);
    }

    hashTable1.printTable();
    hashTable2.printTable();
    return 0;
}
```

3. 调试分析

程序调试中遇到的问题及解决方法

1. **线性探测中的循环问题**：
   - 问题：在使用线性探测时，数组的访问超出了表的大小，导致访问非法内存。
   - 解决方法：在探测时使用模运算，确保索引始终在表的大小范围内。
2. **链表哈希表中的链表操作问题**：
   - 问题：链表插入时，未正确更新链表头指针，导致新插入的元素丢失。
   - 解决方法：确保每次插入时，新的节点被插入到链表的头部，并更新链表头指针。
3. **输出问题**；
   - 问题： 链式哈希表 难以以一目了然的输出格式进行输出
   - 解决方法： 参考输出样例，将输出格式进行转置，使得哈希表地址及其对应的关键 字输出更加美观。

4. 算法的时空分析

时间复杂度

1. **线性探测哈希表**：
   - **插入操作**：最坏情况下，插入的时间复杂度是 O(n)，其中 n 是哈希表的大小，发生在哈希冲突较多的情况下。
   - **查找操作**：最坏情况下，查找的时间复杂度是 O(n)，因为可能需要遍历整个哈希表。
2. **链表哈希表**：
   - **插入操作**：每个链表的插入操作时间复杂度是 O(1)，即使发生哈希冲突，也只需要在链表头部插入。
   - **查找操作**：最坏情况下，查找的时间复杂度是 O(n)，当所有元素都哈希到同一槽位时，链表的查找复杂度为 O(n)。

空间复杂度

1. **线性探测哈希表**：
   - **空间复杂度**：O(n)，需要一个大小为 n 的数组来存储哈希表的数据和一个大小为 n 的布尔数组来标记占用情况。
2. **链表哈希表**：
   - **空间复杂度**：O(n)，每个槽位是一个链表，链表节点需要额外的空间来存储关键字和指向下一个节点的指针。

5. 测试结果及分析

测试数据

输入数据：

```
请输入哈希表大小：11
请输入模数：11
请输入关键字数量：8
请输入关键字：23 35 12 56 123 39 342 90
```

输出：

线性探测哈希表

```
哈希表的地址：   0       1       2       3       4       5       6       7       8       9       10
表中的关键字：   -       23      35      12      56      123     39      342     90      -       -
成功查找次数：   0       1       1       3       4       4       1       7       7       0       0
失败查找次数：   1       9       8       7       6       5       4       3       2       1       1
查找成功的平均查找长度: 3.5
查找失败的平均查找长度: 4.27273
```

链表哈希表

```
哈希表的地址：   表中的关键字：
0       -
1       342 56 12 23
2       90 123 35
3       -
4       -
5       -
6       39
7       -
8       -
9       -
10      -

342对应成功查找次数：    1
56对应成功查找次数：    2
12对应成功查找次数：    3
23对应成功查找次数：    4
90对应成功查找次数：    1
123对应成功查找次数：    2
35对应成功查找次数：    3
39对应成功查找次数：    1

失败查找次数：   1       5       4       1       1       1       2       1       1       1       1
查找成功的平均查找长度: 2.125
查找失败的平均查找长度: 1.72727
```

分析

- 线性探测哈希表在插入元素时，当发生哈希冲突时，需要不断探测其他槽位，导致查找的平均长度增加。
- 链表哈希表通过链表解决冲突，因此查找元素时需要遍历链表，但链表的长度一般较短，查找成功的平均查找长度较小。

6. 实验体会和收获

通过本次实验，我深刻理解了哈希表的两种冲突解决策略：线性探测和链表哈希。线性探测的简单实现使得它在内存上更紧凑，但在处理大量冲突时效率较低。而链表哈希则通过使用链表存储冲突的元素，避免了连续探测的开销，但增加了内存开销。实验加深了我对哈希表效率分析的理解，尤其是不同负载因子对性能的影响。